Fayda API — JSON Integration Guide

Purpose: This guide is for clients that call the Fayda National ID Validation API and want identity data as **JSON**, not a PDF.

Base URL: `https://otp.affiliate.pro.et/api/v1`
Authentication: Send `x-api-key: YOUR_KEY` on every request.

Important: This is a separate JSON verification flow. `/verify-otp` remains the PDF endpoint. Use `/verify-otp-json` when JSON identity data is required.

PDF vs JSON

Step	PDF API	JSON API
Send OTP	POST /send-otp	POST /send-otp (same)
Verify OTP	POST /verify-otp → PDF bytes	POST /verify-otp-json → JSON
Remaining points	X-Remaining-Points header only	remainingPoints in JSON body and header

Do not call both verify endpoints for the same OTP. Each successful verify deducts 1 point.

Flow

```
Client sends FAN
  → POST /send-otp
  → store session
  → OTP is sent to the masked phone

Client sends OTP
  → POST /verify-otp-json with { otp, fanNumber, session }
  → if 200: read data + remainingPoints from JSON
  → if error: show message from JSON
```

Store `session` and `fanNumber` together. Pass `session` back exactly as returned. Do not parse or modify it.

1. Send OTP

```
POST /api/v1/send-otp
Content-Type: application/json
x-api-key: YOUR_KEY

{ "fanNumber": "1234567890123456" }
```

Success (200)

```
{
  "session": "opaque-string-store-as-is",
  "phone": "******7890"
}
```

Field	Description
session	Opaque token. Store as-is and send it unchanged to step 2.
phone	Masked mobile number the OTP was sent to.

2. Verify OTP (JSON)

```
POST /api/v1/verify-otp-json
Content-Type: application/json
x-api-key: YOUR_KEY

{
  "otp": "123456",
  "fanNumber": "1234567890123456",
  "session": "opaque-string-from-step-1"
}
```

Field	Type	Required	Description
otp	string	yes	6-digit code from SMS.
fanNumber	string	yes	Same 16-digit FAN from step 1. Spaces are stripped.
session	string	yes	Exact session from step 1.

Success (200)

```
{
  "success": true,
  "remainingPoints": 497,
  "data": {
    "fanNumber": "1234567890123456",
    "fin": "1234567890",
    "name": {
      "amharic": "...",
      "english": "..."
    },
    "dateOfBirth": {
      "ethiopian": "01/05/1988",
      "gregorian": "1988-05-09"
    },
    "gender": {
      "amharic": "...",
      "english": "Male"
    },
    "nationality": {
      "amharic": "...",
      "english": "Ethiopian"
    },
    "phone": "0912345678",
    "address": {
      "city": { "amharic": "...", "english": "..." },
      "subCity": { "amharic": "...", "english": "..." },
      "woreda": { "amharic": "...", "english": "..." }
    },
    "issueDate": {
      "ethiopian": "...",
      "gregorian": "..."
    },
    "expiryDate": {
      "ethiopian": "...",
      "gregorian": "..."
    },
    "photo": "base64-or-data-uri",
    "qrCode": "base64-or-data-uri"
  }
}
```

Also returned as a header: `X-Remaining-Points: 497`

Field	Description
remainingPoints	Account balance after this request (1 point deducted).
data.fanNumber	FAN you submitted.
data.fin	FIN / UIN.
data.photo	Profile photo (base64).
data.qrCode	QR image (base64).

Error (400 / 401 / 402 / 500)

```
{
  "success": false,
  "message": "Human-readable error"
}
```

Use `response.json()` on this endpoint. Success is JSON, not a PDF.

3. Check Remaining Points (Optional)

```
GET /api/v1/balance
x-api-key: YOUR_KEY
```

Success (200)

```
{
  "success": true,
  "username": "your-username",
  "email": "you@example.com",
  "points": 497,
  "status": "active"
}
```

status	Meaning
active	More than 10 points.
low	1–10 points.
depleted	0 points. Further verify calls return 402.

Node.js (Axios)

```
const axios = require('axios');

const BASE_URL = 'https://otp.affiliate.pro.et/api/v1';
const API_KEY = process.env.FAYDA_API_KEY;

function faydaHeaders() {
  return {
    'Content-Type': 'application/json',
    'x-api-key': API_KEY,
  };
}

async function validateNationalIdJson(fanNumber, otp) {
  const step1 = await axios.post(
    `${BASE_URL}/send-otp`,
    { fanNumber },
    { headers: faydaHeaders() }
  );

  const { session, phone } = step1.data;

  const step2 = await axios.post(
    `${BASE_URL}/verify-otp-json`,
    { otp, fanNumber, session },
    { headers: faydaHeaders(), timeout: 120000 }
  );

  const { remainingPoints, data } = step2.data;
  return { phone, remainingPoints, data };
}

async function getBalance() {
  const res = await axios.get(`${BASE_URL}/balance`, {
    headers: { 'x-api-key': API_KEY },
  });
  return res.data;
}
```

No special `responseType` is needed. Default Axios JSON parsing is correct for `/verify-otp-json`.

Python (requests)

```
import os
import requests

BASE_URL = "https://otp.affiliate.pro.et/api/v1"
API_KEY = os.environ["FAYDA_API_KEY"]
HEADERS = {"x-api-key": API_KEY, "Content-Type": "application/json"}


def validate_national_id_json(fan_number, otp):
    step1 = requests.post(
        f"{BASE_URL}/send-otp",
        json={"fanNumber": fan_number},
        headers=HEADERS,
        timeout=60,
    )
    step1.raise_for_status()
    session = step1.json()["session"]
    phone = step1.json()["phone"]

    step2 = requests.post(
        f"{BASE_URL}/verify-otp-json",
        json={"otp": otp, "fanNumber": fan_number, "session": session},
        headers=HEADERS,
        timeout=120,
    )
    step2.raise_for_status()
    body = step2.json()
    return {
        "phone": phone,
        "remaining_points": body["remainingPoints"],
        "data": body["data"],
    }


def get_balance():
    res = requests.get(
        f"{BASE_URL}/balance",
```

```
        headers={"x-api-key": API_KEY},
        timeout=30,
    )
    res.raise_for_status()
    return res.json()
```

Telegram Bot (JSON)

```
const { Telegraf } = require('telegraf');
const axios = require('axios');

const BASE_URL = 'https://otp.affiliate.pro.et/api/v1';
const API_KEY = process.env.FAYDA_API_KEY;
const bot = new Telegraf(process.env.BOT_TOKEN);
const pending = new Map(); // chatId -> { fanNumber, session }

function faydaHeaders() {
  return {
    'Content-Type': 'application/json',
    'x-api-key': API_KEY,
  };
}

bot.command('start', (ctx) =>
  ctx.reply('Send a 16-digit FAN number to begin.')
);

bot.on('text', async (ctx) => {
  const chatId = ctx.chat.id;
  const text = ctx.message.text.trim();
  const state = pending.get(chatId);

  try {
    if (state?.session && /^\d{6}$/.test(text)) {
      await ctx.reply('Verifying OTP…');

      const res = await axios.post(
        `${BASE_URL}/verify-otp-json`,
        {
          otp: text,
          fanNumber: state.fanNumber,
          session: state.session,
        },
        {
          headers: faydaHeaders(),
          timeout: 120000,
          validateStatus: () => true
        }
      );

      pending.delete(chatId);

      if (res.status !== 200 || !res.data?.success) {
        await ctx.reply(res.data?.message || 'Verification failed.');
        return;
      }

      const { remainingPoints, data } = res.data;
      await ctx.reply([
        data.name.english,
        `FAN: ${data.fanNumber}`,
        `FIN: ${data.fin}`,
        `DOB: ${data.dateOfBirth.gregorian}`,
        `Phone: ${data.phone}`,
        `Remaining points: ${remainingPoints}`,
      ].join('\n'));
      return;
    }

    const fanNumber = text.replace(/\s+/g, '');
    if (!/^\d{16}$/.test(fanNumber)) {
      await ctx.reply(
        'Send a 16-digit FAN, or a 6-digit OTP after requesting one.'
      );
      return;
    }

    const step1 = await axios.post(
      `${BASE_URL}/send-otp`,
      { fanNumber },
      { headers: faydaHeaders() }
    );
```

```
    pending.set(chatId, {
      fanNumber,
      session: step1.data.session
    });

    await ctx.reply(
      `OTP sent to ${step1.data.phone}. Reply with the 6-digit code.`
    );
  } catch (err) {
    await ctx.reply(
      err.response?.data?.message ||
      err.message ||
      'Request failed.'
    );
  }
});

bot.launch();
```

Checklist

• Call /verify-otp-json, not /verify-otp.

• Parse the success body as JSON (response.json() / Axios default).

• Read remaining points from remainingPoints (also in X-Remaining-Points).

• Keep session per request and send it back unchanged.

• Allow ~60–120 seconds timeout on verify; upstream validation can take a few seconds.

• Keep the API key on the server only. Never put it in a mini-app or client.

• Do not also call /verify-otp for the same OTP.

Common Errors

Message / status	Meaning
401 Invalid API key	Wrong or missing x-api-key.
402 Insufficient points	Recharge the API account.
400 otp / fanNumber / session	Missing or malformed field.
session is invalid or malformed	Session was changed, truncated, or from a different send-otp call.
Timeout	Increase client timeout on verify-otp-json.
PDF / invalid JSON	You called /verify-otp instead of /verify-otp-json.

Key rule: If you need identity data as JSON, use `POST /verify-otp-json`. The PDF endpoint and JSON endpoint are separate verification endpoints.